

Empowering Autonomous Debugging Agents with Efficient Dynamic Analysis

JIAHONG XIANG[†], Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, China

XIAOYANG XU, Southern University of Science and Technology, China

XIAOPAN CHU, Southern University of Science and Technology, China

HONGLIANG TIAN, Ant Group, China

YUQUN ZHANG^{†*}, Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, China

Autonomous agents for automated program repair represent a promising frontier in software engineering, yet their effectiveness is often hindered by reliance on post-mortem, coarse-grained execution feedback. While integrating traditional interactive debuggers seems a natural solution, their low-level, line-by-line interaction paradigm turns out to be cost-inefficient for LLM-based agents, leading to exhausted budgets and unproductive loops. To mitigate this, we introduce Agent-centric Debugging Interface (ADI), a novel agent-centric debugging interface designed for cost-efficient, end-to-end autonomous interaction. Specifically, Agent-centric Debugging Interface realizes a function-level interaction paradigm, powered by our *Frame Lifetime Trace*—a comprehensive data structure encapsulating a function’s stateful execution trace—and a set of high-level navigational commands.

Our extensive evaluation on the SWE-bench benchmark demonstrates the effectiveness and efficiency of ADI. By simply equipping a basic agent with ADI, it successfully resolves 63.8% of the tasks on the SWE-bench-Verified set, even slightly outperforming the highly-optimized and high-investment CLAUDE-TOOLS agent, at an average cost of \$1.28 per task with Claude-Sonnet-3.7. Furthermore, we demonstrate ADI’s generality by integrating it as a plug-and-play component into the existing SOTA agents, delivering consistent gains ranging from 6.2% to 18.5% on the resolved tasks. These results indicate that Agent-centric Debugging Interface could achieve a general and efficient enhancement for the existing autonomous agents.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; **Software testing and debugging**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Debugging, Large Language Model, Automated Program Repair

ACM Reference Format:

Jiahong Xiang[†], Xiaoyang Xu, Xiaopan Chu, Hongliang Tian, and Yuqun Zhang^{†*}. 2026. Empowering Autonomous Debugging Agents with Efficient Dynamic Analysis. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE031 (July 2026), 22 pages. <https://doi.org/10.1145/3797126>

*Yuqun Zhang is the corresponding author.

[†]These authors are also affiliated with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China.

Authors’ Contact Information: [Jiahong Xiang[†]](#), Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, Shenzhen, China, xiangjh2022@mail.sustech.edu.cn; [Xiaoyang Xu](#), Southern University of Science and Technology, Shenzhen, China, 12112620@mail.sustech.edu.cn; [Xiaopan Chu](#), Southern University of Science and Technology, Shenzhen, China, 12111712@mail.sustech.edu.cn; [Hongliang Tian](#), Ant Group, Hangzhou, China, tate.thl@antgroup.com; [Yuqun Zhang^{†*}](#), Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, Shenzhen, China, zhangyq@sustech.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE031

<https://doi.org/10.1145/3797126>

1 Introduction

Recently, Large Language Models (LLMs) have demonstrated remarkable capabilities in addressing a wide range of software engineering challenges [20, 22–24, 26, 28, 31, 41, 43–47, 53, 56, 57, 62]. A particularly promising frontier is the development of autonomous agents for automated program repair [16, 17, 30, 32, 41, 48, 50, 54, 58], which are increasingly benchmarked against complex, repository-level real-world tasks like those in the SWE-bench suite [24]. Specifically, these automated program repair tasks are highly demanding, requiring an agent to possess capabilities like issue reproduction, fault localization, and patch generation. In particular, the existing state-of-the-art agents primarily employ an iterative debugging process through self-correction based on execution feedback [19, 41, 50, 54, 58]. However, relying solely on post-mortem [51], coarse-grained execution output for self-correction causes limited effectiveness for fixing complex bugs, which are typically involved in intricate data and control flow relationships that are often not presented in the final execution results. For instance, a simple failure signal (e.g., an assertion error) or a non-crashing unexpected behavior (e.g., an incorrect output) caused by a complex bug often reveals little information about the state-dependent execution path leading to the failure, i.e., providing insufficient assistance for LLMs to locate the bug and further generate the patch accordingly, thus compromising the power of applying LLMs [25, 29, 61].

To mitigate the above-mentioned issue, one can be inspired by the existing human-in-the-loop techniques, i.e., using interactive debuggers such as GDB [42], PDB [40], or those integrated into IDEs, to inspect a program's state and understand its complex data and control flow. Existing research attempts to integrate agents with these traditional, human-in-the-loop tools for both enabling fine-grained dynamic analysis and reducing the need for manual intervention, e.g., building interfaces where an agent assists human developers by answering high-level queries through autonomous control of a debugger [29] and developing interactive, text-based environments where, guided by a developer-written set of test cases, an agent is provided with debugger access to train and evaluate its ability to autonomously repair code [61]. Ideally, an agent could leverage these debuggers to efficiently resolve complex debugging tasks with the assistance from the underlying human developer skills. However, while it is natural that the existing human-in-the-loop debuggers operate on a line-by-line basis using low-level, atomic commands (e.g., `next` and `print var`), such a paradigm renders the usage of an LLM agent rather cost-inefficient, thus making the agent-debugger integration somewhat ineffective in the real world. Specifically, each atomic command provides only a sliver of state information but incurs the substantial cost of a complete LLM inference cycle. When attempting to operate fully autonomously, agents easily exhaust their computational budget or halt in unproductive debugging loops, preventing them from reaching a solution [8, 61]. As a result, existing techniques can hardly be fully autonomous, i.e., they have to rely on workarounds such as human-in-the-loop guidance [29] or operate under strong assumptions such as high-quality assertions and test cases [25, 33, 61], posing a pressing need for agent-centric interactive debugging interfaces.

In this paper, we introduce Agent-centric Debugging Interface (ADI) to realize cost-efficient end-to-end autonomous agent-debugger interaction, freeing agents from reliance on human-in-the-loop guidance. Our approach is established on two key designs. First, instead of the traditional statement-level agent-debugger interaction, ADI realizes the function-level agent-debugger interaction via the *Frame Lifetime Trace* (FLT), a comprehensive data structure that encapsulates the stateful execution trace of a single function invocation. Specifically, an FLT contains the function's arguments, its final return value, a complete, ordered trace of all executed statements along with their resulting state modifications, and identifiers linking to its caller and any downstream callees. Second, ADI equips an agent with a set of high-level navigational commands designed for efficient exploration of a

program's dynamic states. These commands empower the agent to strategically control execution flows by setting function-level conditional breakpoints and gain a high-level overview of program structures by navigating the associated dynamic call graphs.

We perform an extensive study to evaluate the effectiveness and efficiency of ADI. Our evaluation shows that by simply equipping a basic agent with ADI, the resulting agent, FRAMEPILOT, achieves superior performance. Specifically, it resolves 63.8% of tasks on the SWE-bench Verified benchmark [36], even slightly outperforming the highly-optimized and high-investment CLAUDE-TOOLS agent, at an average cost of \$1.28 per task with Claude-Sonnet-3.7 [12]. To demonstrate the generality of ADI, we integrate it as a plug-and-play component into two SOTA agents with distinct architectures, mini-SWE-agent [24] and AUTOCODEROVER [41]. This integration delivers consistent performance gains ranging from 6.2% to 18.5% in terms of the resolved tasks at a modest cost, showcasing ADI's utility as a general-purpose enhancement.

In summary, this paper makes the following contributions:

- **Agent-centric Debugging Interface.** We propose Agent-centric Debugging Interface, a novel interactive debugging interface designed for autonomous agents that enables cost-efficient dynamic analysis through a function-level interaction model, powered by our *Frame Lifetime Trace* and a set of high-level navigational commands.
- **Superior Performance.** Our extensive evaluation on the SWE-bench benchmark shows that by simply equipping a basic agent with Agent-centric Debugging Interface, it achieves superior performance, resolving 63.8% of tasks in SWE-bench Verified, even slightly outperforming the highly-optimized and high-investment CLAUDE-TOOLS agent, at an average cost of \$1.28 per task with Claude-Sonnet-3.7.
- **General-Purpose Enhancement.** We demonstrate the generality of ADI by integrating it as a plug-and-play component into two SOTA agents with distinct architectures, achieving consistent performance gains from 6.2% to 18.5% at a modest cost. This indicates that ADI could potentially enhance the performance of general agent architectures for automated program repair tasks.

2 Motivating Example

```
Modeling's `separability_matrix` does not compute separability correctly for nested
CompoundModels

It's separability matrix as you might expect is a diagonal:
>>> separability_matrix(m.Pix2Sky_TAN() & m.Linear1D(10) & m.Linear1D(5))
array([[ True,  True, False, False],
       [ True,  True, False, False],
       [False, False,  True, False],
       [False, False, False,  True]])

The output matrix is again, as expected, the outputs and inputs to the linear models
are separable and independent of each other.

If however, I nest these compound models:
>>> separability_matrix(m.Pix2Sky_TAN() & cm)
array([[ True,  True, False, False],
       [ True,  True, False, False],
       [False, False,  True,  True],
       [False, False,  True,  True]])

Suddenly the inputs and outputs are no longer separable?
This feels like a bug to me, but I might be missing something?
```

Fig. 1. A real-world automated program repair task from the SWE-bench (astropy-12907 [4])

This section presents a motivating example using a real-world automated program repair task from the SWE-bench dataset, `astropy-12907` [4] (with its issue description shown in Figure 1). Specifically, the task aims to fix the unexpected behavior of the `separability_matrix` function in the Astropy [5] library, which determines if a model’s inputs and outputs are independent. While the function behaves as expected for simple composite models, it produces an incorrect result for nested composite models (colored in red), falsely indicating that their inputs and outputs are entangled. This task is challenging as it is a silent logical error with no clear failure signal, requiring an agent to diagnose it not by the final output, but by inspecting the intermediate states of the recursive calculation process to form hypotheses for the program’s dynamic behavior.

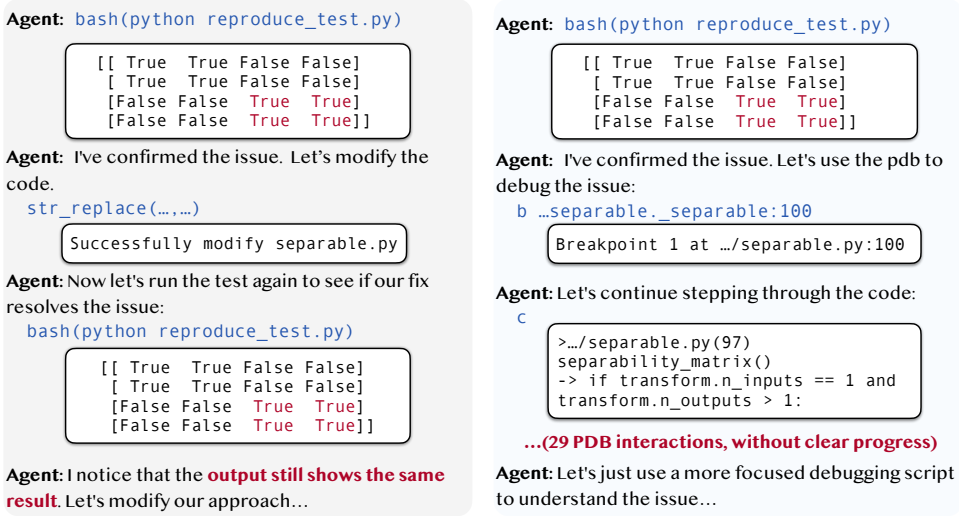


Fig. 2. BaseAgent’s and BaseAgent_{pdb}’s debugging processes on the `astropy-12907` task.

To solve this task, we first employ an agent with the advanced LLM Claude-Sonnet-3.7 that performs post-mortem debugging using only final execution outputs. As shown in Figure 2 (left), this agent halts in the cycle of reasoning and repeated test executions. Specifically, after confirming the issue and applying a blind patch, it re-runs the test only to observe that the “output still shows the same result”. This indicates that having little information about the intricate execution path prevents the agent from understanding the root cause and leads to generating failed patches.

Next, we equip an agent with a standard PDB interface to issue low-level, atomic commands. However, as illustrated in Figure 2 (right), this approach introduces a severe efficiency bottleneck. The agent is forced to issue a long sequence of low-level commands (e.g., `b` (break), `n` (next)) simply to concentrate on the suspicious code region. This process is proven highly inefficient, as each command triggers a costly LLM inference while yielding only a fragment of the program’s state, eventually wasting the agent’s effort after 29 unproductive rounds of interactions.

Ideally, an autonomous agent could resolve this task with the high efficiency and precision illustrated in Figure 3. Rather than the tedious line-by-line stepping, such an agent would first capture a high-level summary of the top-level `_separable` function’s entire execution. By analyzing this summary and the corresponding call tree, the agent can deduce that while correct data flows into the nested `_cstack` function, the matrix it returns is corrupted, thus pinpointing it as the source of the error. To confirm this hypothesis, the agent would then bypass irrelevant execution paths by setting a conditional breakpoint on `_cstack`. This enables the inspection precisely at the

Agent: Let's look at the call graph to understand the execution flow:

call-tree

```
[call-tree] The downstream 3 levels call tree for .../separable.py:_separable#1 is as follows:
└─ Call: def _separable(transform):
    │   └─ Call: def _calculate_separability_matrix(self):
    │       │   Frame ID: .../core.py:Model._calculate_separability_matrix#1
    │       │   ...
    │       └─ Call: def _cstack(left, right):
    │           │   Frame ID: .../separable.py:_cstack#2
    │           │   Argument value: left = array([[1., 1.], [1., 1.]]) , right = array([[1., 0.], [0., 1.]])
    │           │   Return: return np.hstack([cleft, cright])
    │           │   Return value: array([[1., 1., 0., 0.], [1., 1., 0., 0.], [0., 0., 1., 1.], [0., 0., 1., 1.]])
```

Agent: Let's set a breakpoint in the `\_cstack` function with a condition to check when it's handling the nested compound model:

break: .../separable.py:_cstack

condition: isinstance(left, np.ndarray) and left.shape == (2, 2) and isinstance(right, np.ndarray) and right.shape == (2, 2)

```
Current focus frame: .../separable.py:_cstack#2
Called by: .../separable.py:_separable#1
Argument value: left = array([[1., 1.], [1., 1.]]) , right = array([[1., 0.], [0., 1.]])

call      219 def _cstack(left, right):
    │   ...
    line    241     if isinstance(right, Model):
    line    244         cright = np.zeros((noutp, right.shape[1]))
    New var: cright = array([[0., 0.], [0., 0.], [0., 0.], [0., 0.]])
    line    245         cright[-right.shape[0]:, -right.shape[1]:] = 1
    Modified: cright = array([[0., 0.], [0., 0.], [1., 1.], [1., 1.]])
    line    247     return np.hstack([cleft, cright])
    return    247     return np.hstack([cleft, cright])
Return value: array([[1., 1., 0., 0.], [1., 1., 0., 0.], [0., 0., 1., 1.], [0., 0., 1., 1.]])
```

**Pinpoint
Suspicious Code**

Agent: Let's examine the result of the `\_cstack` function more closely:

execute: print(f'cleft:\n{cleft}\ncright:\n{cright}\nresult:\n{np.hstack([cleft, cright])}')

lineno: .../separable.py:_cstack#2:247

```
cleft: [[1. 1.] [1. 1.] [0. 0.] [0. 0.]], cright: [[0. 0.] [0. 0.] [1. 1.] [1. 1.]
```

Agent: **I've identified the issue.** In the `\_cstack` function, when handling arrays (which happens when we have nested compound models), it sets the values in 'cright' to 1 instead of using the actual values from the 'right' array: `cright[-right.shape[0]:, -right.shape[1]:] = 1` #<-- This should use the values from right

Fig. 3. An Efficient Debugging Process on the astropy-12907 Task.

suspicious code region, where a targeted query provides unambiguous evidence of the root cause: the cright array is incorrectly filled with ones. With this confirmation, the agent can formulate the correct patch, resolving the issue with the same logic as the official developer-written solution. This motivating example illustrates the value of a well-designed agent-debugger interface.

3 Background & Related Work

3.1 Traditional Debuggers & Function Frame

Traditional interactive debuggers, such as GDB [42] and PDB [40], provide a powerful, human-in-the-loop interface for investigating program behaviors. As in Figure 4 (left), their designs are based on a Read-Eval-Print Loop (REPL) where a developer issues a sequence of low-level, atomic commands like step, next, and print to trace a program's execution path with one statement at a time.

The underlying data structure that enables this statement-level inspection of the program's state is the program's call stack which tracks all active function calls. Each time a function is invoked, the

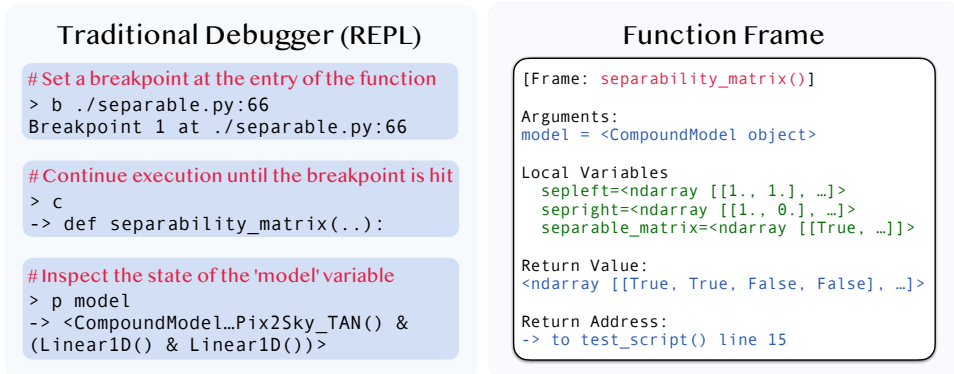


Fig. 4. Traditional Debugger REPL Interaction and the Function Frame.

corresponding function frame, i.e., activation record, is pushed onto the top of the stack. As depicted in Figure 4 (right), this frame encapsulates the essential execution context for that specific function call, containing its arguments, local variables, and the return address to its caller. The associated Last-In, First-Out (LIFO) structure indicates that the stack of frames creates a nested record of the execution path, with the top frame always representing the ongoing executing function. Such a design reveals that the function frame is essentially a natural, higher-level unit of abstraction, potentially representing a function’s execution context. This insight positions the function frame as the natural focal point for an agent-centric debugging interface.

3.2 Agent-based Debugging

Researchers have explored various approaches to enhance the debugging capabilities of LLM-based agents, with a prominent focus on iterative self-correction from execution feedback. Specifically, Self-Debugging [19] teaches an agent to perform rubber duck debugging by comparing natural language explanations of its code against the execution outcome. Similarly, ChatRepair [54] establishes a conversational repair loop where the agent learns from test failures to generate a more accurate patch. RepairAgent [17] further extends this by treating the LLM as an autonomous agent that freely decides which tool to invoke next, interleaving actions like gathering information and attempting a fix based on feedback from prior attempts. Nowadays, the idea of self-correction established by these works has become the primary debugging paradigm for many state-of-the-art autonomous agents [17, 41, 50, 58]. Related work has also explored adjacent directions such as execution-trace-driven repair [18], conversational debugging assistants [15], and notebook-based error-resolution agents [21]. Another group of work utilizes fault localization (FL) techniques to narrow down the search space for a bug before a repair is attempted, e.g., agents like AUTOCODEROVER [41] can optionally leverage Spectrum-Based Fault Localization (SBFL) [9] to prioritize searching within suspicious code regions identified by test execution coverage. However, the effectiveness of these approaches is limited by their core dependencies, i.e., self-correction on post-mortem, coarse-grained execution output and fault localization on precise trigger tests, which prevent the agent from interactively exploring the program’s dynamic state space.

To overcome the above-mentioned limitations of post-mortem, coarse-grained debugging analysis, one alternative solution is to equip agents with interactive debuggers to access fine-grained, dynamic program state. For instance, ChatDBG [29] enables a collaborative dialogue where an

LLM agent autonomously controls standard debuggers like GDB and PDB to investigate a programmer's high-level queries. To facilitate the development of such agents, debug-gym [61] provides a text-based interactive environment for agents to learn how to use debugging tools for code repair. Building on this, other works have applied interactive debugging to specific domains or with structured methodologies. EnIGMA [8] introduces Interactive Agent Tools to wrap command-line utilities like GDB for solving complex cybersecurity tasks. AutoSD [25] employs a scientific debugging approach, where the agent forms hypotheses and then uses a debugger to conduct experiments to validate them. Similarly, VulDebugger [33] treats debugging as a state-comparison problem, using a debugger to continuously compare the program's actual runtime state against an expected correct state to repair vulnerabilities. However, simply integrating these human-centric debuggers with automated software engineering agents introduces an inherent mismatch. These debuggers operate on a Read-Eval-Print Loop (REPL), requiring a sequence of low-level, atomic commands (e.g., next, step, print var). For an autonomous agent, this interaction model is highly inefficient, where the costly LLM inference of each interaction round on a low-level command yields only a fragment of the program's state, eventually making the debugging process overwhelmingly expensive and compromising the agent's power on autonomously fixing complex bugs in an end-to-end manner. As a result, there is an urgent need for agent-centric interactive debugging interfaces which can facilitate the efficient retrieval of semantically rich, high-level summaries of program state, aligning the debugging process with the inferential capabilities of LLM agents.

4 Agent-centric Debugging Interface

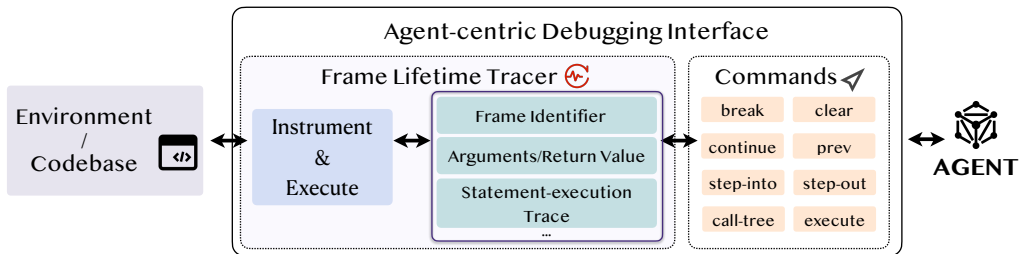


Fig. 5. Agent-centric Debugging Interface Framework

In this section, we introduce Agent-centric Debugging Interface (ADI) to realize cost-efficient end-to-end autonomous function-level agent-debugger interaction, freeing agents from reliance on human-in-the-loop guidance. Figure 5 provides an overview of this framework, where a *Frame Lifetime Tracer* instruments and executes a codebase to generate a series of *Frame Lifetime Traces* (FLT). Each FLT is a self-contained data structure that captures the complete lifetime of a single function invocation, with its arguments, return value, and a detailed trace of all internal state modifications. An agent then inspects and navigates using a set of high-level *Commands*. In particular, we model the interactive debugging session as a state transition system. The states in this system are defined over our core data abstraction, i.e., the *Frame Lifetime Trace* (FLT). The transitions are triggered by a set of agent-centric commands with well-defined semantics. These components are unified by the ADI Interaction Algorithm to realize the agent-driven dynamic analysis for automated program repair. Importantly, ADI does not pre-compute a complete statement-level trace for an entire execution. Instead, it first records a lightweight function-level frame sequence and constructs a detailed *Frame Lifetime Trace* only for the frame currently selected by the inspection index.

4.1 Frame Lifetime Trace

The core abstraction of ADI is the *Frame Lifetime Trace* (FLT). It is a self-contained data structure that captures the complete lifetime of a single function invocation, from its entry to its exit. This allows the agent to reason about a function's behavior holistically, rather than statement by statement. Formally, an FLT is defined as a tuple:

$$FLT = (fid, fid_{caller}, args, ret, \tau)$$

Where each component is defined as:

- ***fid* (Frame Identifier)**: A unique identifier for the target function invocation. It is constructed as a pair $(func, index)$, where *func* is the fully-qualified [39] function name and *index* is an invocation index. For example, $(my_class:my_func, 3)$ refers to the invocation of *my_func* during the execution for the third time.
- ***fid_{caller}* (Caller Frame Identifier)**: The frame identifier (*fid*) of this frame's immediate caller. This is crucial for navigating the call stack. For the entry-point frame, this can be null.
- ***args* (Arguments)**: A key-value map of the function's formal parameter names to their corresponding values at the snapshot of invocation.
- ***ret* (Return Value)**: The value returned by the function upon its exit. If the function terminates upon an unhandled exception, this component may contain exception information.
- ***τ* (Statement-execution Trace)**: An ordered trace of all statements executed within the function's lifetime. The trace is a sequence of *execution steps*, where each step is formally defined as the tuple:

$$step = (lineno, stmt, \Delta S, fid_{callee})$$

- *lineno* (Line Number): The line number of the executed statement in the source code.
- *stmt* (Statement): The source code of the statement itself.
- ΔS (State Modifications): A set of state modifications caused by this statement. Each modification is defined as a tuple $(variable, old_value, new_value)$, capturing the definition of new variables or changes to existing ones. If a statement causes no state change, ΔS is empty.
- *fid_{callee}* (Callee Frame Identifier): If the statement invokes a function, this field contains the identifier (*fid*) of the resulting frame (the callee). Otherwise, it is null.

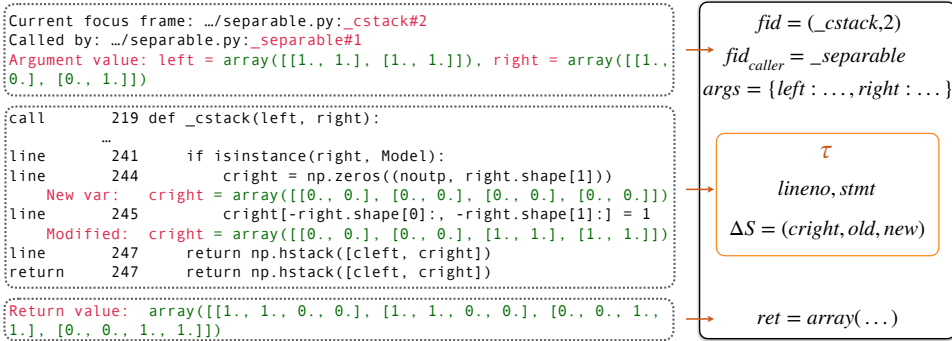


Fig. 6. The *Frame Lifetime Trace* (FLT) of the `_cstack#2` invocation from the `astropy-12907` task

Figure 6 presents an FLT instance for the `_cstack#2` invocation from our motivating example, illustrating the mapping between its textual representation (left) and its formal components (right). Note that the function's representative invocation context, including its caller and arguments,

is mapped to the *fid*, *fid_{caller}*, and *args* components. The statement-execution trace (τ) records the execution sequence and presents the resulting state modifications for each statement (ΔS). For instance, the sequence shows the *cright* variable being created (New var) at line 244 and subsequently modified (Modified) at line 245. Finally, the function's exit value is encapsulated in the *ret* component.

We then model the interactive debugging session as a state transition system based on the FLT concept. The complete state of a debugging session is captured by a tuple Σ :

$$\Sigma = (B, T, i)$$

with the parameters illustrated as follows.

- ***B* (Breakpoint Set)**: A set of pairs (*func*, *condition*), where *func* is a function identifier and *condition* is an optional boolean expression evaluated at the function entry (by default True).
- ***T* (Full Execution Trace)**: An ordered sequence $\langle FLT_0, FLT_1, \dots, FLT_n \rangle$ representing all function calls from a single program execution, ordered by invocation time.
- ***i* (Inspection Index)**: An index $i \in \{0, \dots, n\}$, that identifies the ongoing frame FLT_i under the agent's inspection.

Then, the commands in the following section are defined as functions that map a state Σ to a new state Σ' , denoted $\Sigma \rightarrow \Sigma'$.

4.2 Agent-Centric Debugging Commands

The ADI command set equips the agent with a structured interface to explore the program's execution trace *T*. First, the agent uses *break* and *clear* to dynamically manage a set of function-level conditional breakpoints, defining which execution frames are points of interest. Once these points are established, the agent can navigate the program's dynamic state. The *continue* and *prev* commands enable chronological traversal along the execution timeline, moving between breakpoint hits, while *step-out* and *step-into* serve as their respective reverse operations to navigate the call stack between caller and callee frames. This bidirectional navigation allows the agent to freely access the execution history. Finally, for deeper inspection without altering its position, the agent can use query commands: *call-tree* provides a local overview of the call hierarchy, and *execute* facilitates "what-if" analysis by running a statement under a specific context. Each command transforms the debugging state Σ by either updating the breakpoint set *B* or the inspection index *i*.

- ***break(func, [condition])***: Adds a breakpoint to the set *B*. The optional *condition* is a boolean expression evaluated at the function entry or exit, with the default value True. The state transition is

$$\Sigma \rightarrow \Sigma' \quad \text{where } \Sigma' = (B \cup \{(func, condition)\}, T, i).$$

This command allows the agent to register specific function invocations as points of interest for subsequent navigation. Specifically, this operation updates the state solely by adding the new (*func*, *condition*) pair to the breakpoint set *B*.

- ***clear(func, [condition])***: The inverse command of *break*, which removes a specified breakpoint from the set *B*.
- ***continue()***: Advances the inspection index forward to the next frame that matches a registered breakpoint. In particular, we set *i* as the current inspection index. First, the set *J* collects the indices *j* of all subsequent frames ($j > i$) that match a breakpoint. Then the new index *i'* is set to the first of these matches ($\min(J)$), or remains unchanged if no future match exists.

$$\Sigma \rightarrow \Sigma' \quad \text{where } \Sigma' = (B, T, i'),$$

with i' determined as:

$$J = \{j \mid j > i \wedge (T[j].fid.func, condition) \in B \wedge evaluate(T[j].args, condition)\},$$

$$i' = \begin{cases} \min(J) & \text{if } J \neq \emptyset, \\ i & \text{otherwise.} \end{cases}$$

After executing this command, the agent receives the FLT $T[i']$.

- **prev():** The inverse command of `continue`, which moves the inspection index backward to the most recent preceding breakpoint match and returns the corresponding frame $T[i']$.
- **step-into(fid):** This command provides targeted navigation, moving the inspection index directly to a frame specified by its unique identifier (`fid`). It allows the agent to “step into” a specific function call it has identified, such as a callee discovered from the current context, without needing to traverse the execution timeline sequentially. Specifically, this operation locates the frame with the matching `fid` within the entire trace T and updates the current inspection index i to the found frame’s index j ; the index remains unchanged if the `fid` is not found. The state transition is

$$\Sigma \rightarrow \Sigma' \quad \text{where } \Sigma' = (B, T, i'),$$

with i' determined as:

$$i' = \begin{cases} j & \text{if } \exists j, T[j].fid = fid, \\ i & \text{otherwise.} \end{cases}$$

After executing this command, the agent receives the FLT $T[i']$.

- **step-out():** The inverse command of `step-into`, which moves the inspection index up the call stack from the current frame to its direct caller, returning the corresponding frame $T[i']$.
- **call-tree():** This query command returns a tree-like summary of the downstream call hierarchy from the current frame, allowing the agent to preview subsequent execution paths. Each node in the tree represents a function call, annotated with its signature, `fid`, arguments, and return value. This command is read-only and does not change the debugging state ($\Sigma' = \Sigma$).
- **execute(fid, stmt, lineno, k):** This command provides a powerful “what-if” analysis capability by dynamically executing a statement within a specific execution context. It is designed for fine-grained state inspection, such as checking a variable’s value on a particular loop iteration without setting a complex breakpoint. Upon invocation, the command re-executes the program to reach the target frame identified by `fid` and the k -th visit to line `lineno`. It then temporarily injects `stmt` in that execution context and returns the resulting output. This operation does not modify the source code or the canonical trace T .

4.3 ADI Interaction Algorithm

Algorithm 1 presents a detailed procedure that integrates *Frame Lifetime Traces* with agent-driven interaction. It consists of two cooperating components: the `FRAMELIFETIME TRACER` and the `ADI_INTERACTION_LOOP`. The `ADI_INTERACTION_LOOP` serves as the main engine that processes the interactive navigation commands defined in Section 4.2, while the `FRAMELIFETIME TRACER` is a specialized procedure invoked on-demand to construct a *Frame Lifetime Trace*.

Specifically, `FRAMELIFETIME TRACER` constructs a *Frame Lifetime Trace* for a single function invocation by applying statement-level instrumentation to the selected frame. When the function is invoked, it records the frame identifier (`fid`), the caller frame identifier (`fidcaller`), and the arguments (`args`) (lines 2-4). During the lifetime of the frame, the procedure repeatedly processes the statements executed in the function body (line 6). The `NEXT_STMT` returns the next statement together with its line number (`stmt`, ℓ). Before and after executing the statement, `RECORDENV` captures the ongoing variable environment (lines 7 and 9). The difference between the two environments yields the

Algorithm 1 ADI Interaction Algorithm**Require:** Program P , Input data I

```

1: function FRAMELIFETIMETRACER( $fid$ ,  $P$ ,  $I$ )
2:    $flt \leftarrow \text{NEWFLT}(fid)$ 
3:    $flt.fid_{caller} \leftarrow \text{CALLERFID}$ 
4:    $flt.args \leftarrow \text{ARGSATENTRY}$ 
5:    $\epsilon \leftarrow \text{null}$  ▷ Initialize exception info
6:   while ( $stmt, \ell \leftarrow \text{NEXTSTMT}(fid)$ ) do ▷ Statement-level instrumentation
7:      $Env_{before} \leftarrow \text{RECORDENV}$ 
8:     ( $fid_{callee}, \epsilon$ )  $\leftarrow \text{EXECSTMT}(stmt, P, I)$ 
9:      $Env_{after} \leftarrow \text{RECORDENV}$ 
10:     $\Delta S \leftarrow \text{DELTA STATE}(Env_{before}, Env_{after})$  ▷ Compute state modifications
11:     $flt.\tau.\text{APPEND}((\ell, stmt, \Delta S, fid_{callee}))$ 
12:    if  $\epsilon \neq \text{null}$  then
13:      break
14:    if  $\epsilon \neq \text{null}$  then
15:       $flt.ret \leftarrow \epsilon$  ▷ Return exception info
16:    else
17:       $flt.ret \leftarrow \text{RETURNVALUE}$ 
18:    return  $flt$ 

19: function ADI_INTERACTION_LOOP( $P$ ,  $I$ )
20:    $T \leftarrow \text{BUILDFRAMESEQUENCE}(P, I)$  ▷ For efficiency, only sequence of  $fids$  is collected
21:    $\Sigma \leftarrow (B = \emptyset, T = T, i = 0)$ 
22:   while  $cmd \leftarrow \text{RECEIVECMD}$  do
23:      $\Sigma' \leftarrow \text{APPLYTRANSITION}(\Sigma, cmd)$  ▷ Update debugging state
24:      $\Sigma \leftarrow \Sigma'$ 
25:      $fid_{insp} \leftarrow T[i].fid$ 
26:      $flt_{insp} \leftarrow \text{FRAMELIFETIMETRACER}(fid_{insp}, P, I)$  ▷ Construct FLT of the inspection frame
27:      $out \leftarrow \text{DERIVERESPONSE}(cmd, \Sigma, flt_{insp})$ 
28:      $\text{SENDTOAGENT}(out)$ 

```

state modification ΔS (line 10), which represents the updates caused by the statement. The routine `EXECSTMT` executes the current statement, returning the identifier of a callee frame (fid_{callee}) if the statement invokes a function, and updates the exception variable ϵ if an unhandled exception occurs. Each execution step is thus represented as a tuple $(\ell, stmt, \Delta S, fid_{callee})$ and appended to the statement-execution trace τ . When the frame terminates, if $\epsilon \neq \text{null}$, the return field is set to the exception information; otherwise, it is set to the normal return value (lines 14-17). As a result, an FLT $(fid, fid_{caller}, args, ret, \tau)$ summarizing the invocation is completed (line 18).

The procedure `ADI_INTERACTION_LOOP` in Algorithm 1 manages the interactive debugging session. It begins by executing the program under lightweight function-level instrumentation to initialize the sequence of $fids$ (line 20). The debugging state is initialized as $\Sigma = (B = \emptyset, T = T, i = 0)$, where B is the breakpoint set, T is the frame sequence, and i is the inspection index (line 21). The loop then repeatedly accepts commands from the agent (line 22). Each command is processed according to the state-transition rule of Section 4.2, yielding an updated state Σ' (lines 23-24). From

the new inspection index i , `ADI_INTERACTION_LOOP` retrieves the identifier $fid_{insp} = T[i].fid$ and invokes `FRAMELIFETIMETRACER(fid_{insp})` (lines 25-26) to construct the corresponding FLT. It then derives the response such as the new inspection frame based on Σ and the constructed FLT, and returns it to the agent (lines 27-28). With such a loop, the agent issues a navigation command, and in response, the ADI dynamically constructs a rich, detailed view (the FLT) of the new inspection frame. This on-demand analysis ensures that the agent receives deep contextual information precisely when and where it is needed for the next decision.

5 Evaluation

In this section, we investigate the effectiveness and characteristics of Agent-centric Debugging Interface (ADI). In particular, we build an agent equipped with ADI, namely `FRAMEPILOT`. We attempt to answer the following research questions:

- **RQ1:** *How effective is ADI in advancing automated program repair?* To answer this RQ, we evaluate `FRAMEPILOT` and compare its performance against other baselines.
- **RQ2:** *How do the state-of-the-art agents equipped with ADI perform?* To answer this RQ, we integrate ADI into mini-SWE-agent [58] and `AUTOCODEROVER` [41] and compare the performance of these enhanced agents against their original versions to evaluate ADI's generality as a plug-and-play enhancement.
- **RQ3:** *What are the behavioral characteristics of ADI?* To answer this RQ, we analyze the behavioral characteristics of `FRAMEPILOT` to understand how ADI shapes its problem-solving strategies.

5.1 Benchmark and Evaluation Metrics

We adopt the popular SWE-bench dataset [24] to assess how the Agent-centric Debugging Interface enhances an agent's performance on real-world automated program repair tasks. In particular, we focus on the widely-used SWE-bench Lite [24] and Verified [36] versions, containing 300 and 500 tasks, respectively. These benchmarks employ developer-written unit tests to verify the correctness of agent-generated patches, ensuring a rigorous assessment of the agent's performance.

Following prior work [41, 52, 58], our major evaluation metric is the **Resolved Rate (%)**, reported as **Pass@1** efficacy. A task is considered resolved if a single generated patch successfully applies to the codebase and passes all developer-written acceptance tests. Crucially, these acceptance tests are held out and not used by the agent during the patch generation process to ensure a fair evaluation. Specifically, to ensure consistency and reproducibility, all evaluations are conducted using the official SWE-bench Docker environment provided by the SWE-bench team [49]. Moreover, we report the average API inference cost (**\$ Avg. Cost**) and **Correct Location Rate (%)**, i.e., the percentage of generated patches covering the ground-truth edit locations at the file, hunk, and line granularities [34, 52].

5.2 Baselines and LLMs

Baselines. We first adopt six state-of-the-art agents and approaches as our baselines: `CLAUDE TOOLS` [12], `SWE-agent` [58], mini-SWE-agent [49], `AUTOCODEROVER` [41], `SWESYNINFER` [34], and `AGENTLESS` [52]. In particular, we primarily utilize the performance results reported in their original publications [34, 41, 52, 58] following [34] and the official SWE-bench leaderboard [24]. For our generality evaluation in RQ2, we integrate ADI into two agents selected for their architectural diversity and suitability as integration targets. First, we select mini-SWE-agent as a representative of the ReAct [60]-based agents. Its lightweight and open design makes it an ideal testbed for integrating and evaluating components like ADI. In contrast, `AUTOCODEROVER` employs a fundamentally different retrieve-and-generate strategy. This selection allows us to verify whether the benefits of

ADI are not confined to a single agent design but can serve as a general-purpose enhancement across diverse agent architectures.

We also design two variants of FRAMEPILOT for further ablation studies on Agent-centric Debugging Interface: BaseAgent and BaseAgent_{pdb}. Specifically, BaseAgent represents a standard *post-mortem debugging* approach and equips only with basic file editing and execution tools. BaseAgent_{pdb} extends BaseAgent with a conventional PDB interface, representing a naive integration of *human-in-the-loop interactive debugging*.

LLMs. We adopt four advanced LLMs: Claude-Sonnet-3.7 (claude-3-7-sonnet-20250219) [12], Claude-Sonnet-3.5 (claude-3-5-20241022) [11], GPT-4o (gpt-4o-2024-11-20) [35], and Qwen3 (qwen3-32b) [10]. We obtain the open-source model Qwen3 from Hugging Face [2] and access Claude-Sonnet-3.7, Claude-Sonnet-3.5, GPT-4o through the APIs provided by Anthropic [13] and OpenAI [3]. Inference for the open-source model Qwen3 is conducted on servers with 128-core 2.6GHz AMD EPYC™ ROME 7H12 CPU, 512 GiB RAM, and eight NVIDIA A100 80GB GPUs, running Ubuntu 20.04.6 LTS, utilizing vllm [27] inference framework.

5.3 Implementation and Setups

5.3.1 Agent-centric Debugging Interface. A key challenge in implementing Agent-centric Debugging Interface is the prohibitive performance overhead of full-program, statement-level tracing. To mitigate this, we leverage the `sys.settrace` facility in Python for selective instrumentation. Instead of tracing the entire program, we activate fine-grained, statement-level tracing on-demand, only for the single frame currently designated by the agent’s inspection index, thus providing deep visibility precisely where needed, without incurring the cost of global instrumentation. Specifically, we measure the runtime overhead of our FRAMELIFETIME TRACER on SWE-bench Verified, observing a modest increase in average execution time from 0.68s to 0.87s when executing the official failing tests with tracing on the buggy function frames. Moreover, following the SWE-bench protocol [1], developer-written failing tests are held out, and the agent instead generates a reproduction script from the issue description for ADI-based debugging.

We also realized several targeted optimizations. For frames containing large loops, our tracer intelligently captures only the first and last iterations, reporting the intermediate ones as a single “skipped” block to the agent. The `call-tree` command is also bounded, exploring only three levels of the callee hierarchy to prevent the agent from being overwhelmed with excessive data. Finally, to support the `execute` command, we implemented code injection via Python’s `exec` function. Ensuring that the injected code’s side effects are correctly reflected in the running program requires synchronizing the frame’s local variables using the CPython API call `PyFrame_LocalsToFast` [38], which we access through the `ctypes` library [37]. This specific mechanism is crucial for reliable state manipulation during a debugging session.

5.3.2 FRAMEPILOT and its variants. FRAMEPILOT, BaseAgent, and BaseAgent_{pdb} are built upon the ReAct framework and the same workflow to ensure a fair comparison of their reasoning capabilities. In this framework, the LLM iteratively generates thoughts to reason about problems and actions to invoke tools, receiving observations from the system to form the next steps. FRAMEPILOT automates tasks through a four-stage process: code orientation, issue reproduction, root-cause analysis, and patch implementation. Crucially, it leverages the function calls provided by our Agent-centric Debugging Interface to perform on-demand dynamic analysis, particularly during the analysis stage. In contrast, BaseAgent is restricted to a standard toolkit for file operations and bash commands, relying solely on analyzing the final output of program executions. For BaseAgent_{pdb}, following prior work [8, 61], we realize a non-blocking PDB session that the agent interacts with and manages

Function: adi**Purpose:** Executes function-level commands via the Agent Debugging Interface(ADI).**Format:**

```

<function_call>
  <function>adi</function>
  <action>adi_action</action>
  <cmd>start_debugging_cmd</cmd>
  <id>frame_id</id>
  <subparams>action_specific_parameters</subparams>
</function_call>

```

Details:

<action>: The ADI action to execute.
 <cmd>: A single Python command to start debugging.
 <id>: Frame ID in format: file_path:method_name#frame_index.
 ...

Available Action Commands:

break: Set function-level breakpoint with optional condition
clear: Clear function-level breakpoint.
step-into: Jump into a specific frame by frame ID
step-out: Go back to the caller frame
continue: Continue to the next breakpoint frame
prev: Go back to the previous breakpoint frame
call-tree: Render 3-level call tree from current focus frame
execute: Execute Python statements at a specific frame, line number and execution index.

Fig. 7. The prompt of ADI used by FRAMEPILOT.

through function calls. To illustrate how Agent-centric Debugging Interface is exposed to the agent, Figure 7 presents the detailed tool definition used by FRAMEPILOT.

All three agents are evaluated under the same conditions. Following the SWE-agent setup [58], we set a \$4 budget per task for cost control. Moreover, we follow the setup of baseline agents [41, 58], with the model's temperature set to 0 to ensure that its output is more deterministic.

5.3.3 Adapting SOTA Agents for ADI. We integrated Agent-centric Debugging Interface into two SOTA agents, mini-SWE-agent and AUTOCODEROVER, with minimal modifications to their native workflows. For mini-SWE-agent, we adapt its primary prompt with ADI usage instructions and introduce an optional dynamic analysis stage to its workflow, empowering it to invoke interactive debugging. For AUTOCODEROVER, we enhanced its context retrieval agent by prompting it to use ADI for collecting richer, dynamic runtime information about program behaviors before attempting a patch. In both cases, the ADI toolset is deployed within the agents' sandboxed environments and exposed through a lightweight API, making it seamlessly available within their existing action spaces. Due to page limits, more implementation details are shown on our GitHub page [7].

5.4 Result Analysis

5.4.1 RQ1: effectiveness of Agent-centric Debugging Interface. As shown in Table 1, equipping a basic agent with ADI leads to highly competitive performance. When using Claude-Sonnet-3.7 on the Verified benchmark, the ADI-enabled agent achieves a 63.8% resolved rate. This result even slightly exceeds the performance of the highly-optimized and high-investment CLAUDE-TOOLS agent (63.2%), which serves as the foundation for Anthropic's commercial Claude Code product [12]. This strong performance extends to the SWE-bench Lite benchmark, where our agent also achieves a top-performing 50.7% resolved rate. Notably, this leading performance is achieved at an average

Table 1. Performance of FRAMEPILOT and Baselines on SWE-bench Lite and Verified benchmarks.

Agent	LLM	% Resolved		Avg. \$ Cost
		Verified	Lite	
CLAUDE-TOOLS [12, 14]	Claude-Sonnet-3.7	316 (63.2%)	-	3.95 [*]
	Claude-Sonnet-3.5	245 (49.0%)	-	3.11 [*]
SWE-agent [58, 59]	Claude-Sonnet-3.7	291 (58.2%)	144 (48.0%)	-
	Claude-Sonnet-3.5	168 (33.6%)	69 (23.0%)	1.62
	GPT-4o	116 (23.2%)	55 (18.3%)	2.53
mini-SWE-agent [49]	Claude-Sonnet-3.7	264 (52.8%)	-	1.43 [*]
	GPT-4o	108 (21.6%)	-	0.98 [*]
AGENTLESS [52]	Claude-Sonnet-3.5	254 (50.8%)	122 (40.7%)	1.19
	GPT-4o	194 (38.8%)	96 (32.0%)	0.70
AUTOCODEROVER [41]	Claude-Sonnet-3.5	231 (46.2%)	-	0.74
	GPT-4o	192 (38.4%)	92 (30.7%)	0.65
SWESYNINFER [34]	Claude-Sonnet-3.5	177 (35.4%)	71 (23.7%)	0.42
	GPT-4o	159 (31.8%)	62 (20.7%)	0.78
	Qwen2.5-instruct	127 (25.4%)	54 (18.0%)	-
FRAMEPILOT	Claude-Sonnet-3.7	319 (63.8%)	152 (50.7%)	1.28
	Claude-Sonnet-3.5	256 (51.2%)	111 (37.0%)	0.58
	GPT-4o	181 (36.2%)	82 (27.3%)	0.93
	Qwen3	157 (31.4%)	63 (21.0%)	-

"-" indicates data is not applicable or publicly available. * indicates API cost is not publicly reported and is calculated from trajectories [49].

cost of only \$1.28 per task. Moreover, we observe that the ADI-enabled FRAMEPILOT uniquely solves two tasks on the entire SWE-bench leaderboard [49] which none of the rest baselines could. All such results indicate that Agent-centric Debugging Interface is essentially powerful to advance the effectiveness of an agent for automated program repair tasks.

Table 2. Performance of FRAMEPILOT and variants on SWE-bench Lite and Verified benchmarks.

Agent	LLM	% Resolved		% Correct Location			Avg. \$ Cost
		Verified	Lite	Line	Function	File	
BaseAgent	Claude-Sonnet-3.7	275 (55.0%)	130 (43.3%)	47.0%	61.6%	77.0%	1.02
	Claude-Sonnet-3.5	222 (44.4%)	92 (30.7%)	44.4%	56.6%	73.6%	0.49
	GPT-4o	163 (32.6%)	72 (24.0%)	35.8%	49.6%	63.0%	0.87
	Qwen3	146 (29.2%)	57 (19.0%)	36.4%	46.0%	64.8%	-
BaseAgent _{pdb}	Claude-Sonnet-3.7	279 (55.8%)	133 (44.3%)	46.4%	60.2%	76.4%	1.62
	Claude-Sonnet-3.5	225 (45.0%)	96 (32.0%)	44.8%	53.8%	72.0%	0.89
	GPT-4o	156 (31.2%)	71 (23.7%)	33.0%	43.6%	56.0%	1.09
	Qwen3	142 (28.4%)	60 (20.0%)	38.0%	46.8%	63.2%	-
FRAMEPILOT	Claude-Sonnet-3.7	319 (63.8%)	152 (50.7%)	51.2%	66.4%	78.8%	1.28
	Claude-Sonnet-3.5	256 (51.2%)	111 (37.0%)	47.8%	58.6%	74.6%	0.58
	GPT-4o	181 (36.2%)	82 (27.3%)	37.2%	51.2%	65.4%	0.93
	Qwen3	157 (31.4%)	63 (21.0%)	39.2%	49.0%	66.8%	-

"-" indicates data is not applicable or publicly available.

Table 2 presents the results of our ablation study where we compare FRAMEPILOT with BaseAgent and BaseAgent_{pdb}. FRAMEPILOT consistently outperforms both baselines across all LLMs and benchmarks. For instance, on SWE-bench Verified, FRAMEPILOT achieves relative performance gains of 7.5% to 16.0% over BaseAgent and 10.6% to 16.0% over BaseAgent_{pdb}. Similar gains are observed on SWE-bench Lite.

The performance gains from ADI are also highly cost-efficient. While FRAMEPILOT incurs a modest cost increase over BaseAgent (e.g., \$0.26 with Claude-Sonnet-3.7), the cost of BaseAgent_{pdb}

Table 3. Performance of SOTA Agents with and without ADI Integration on SWE-bench Verified benchmark

Approach	LLM	#Resolved	Improv. (%)	Avg. Cost (\$)
mini-SWE-agent	Claude-Sonnet-3.7	264	-	1.43
mini-SWE-agent _{ADI}	Claude-Sonnet-3.7	292	10.6	1.57
mini-SWE-agent	GPT-4o	108	-	0.98
mini-SWE-agent _{ADI}	GPT-4o	128	18.5	1.16
AUTOCODEROVER	Claude-Sonnet-3.5	231	-	0.74
AUTOCODEROVER _{ADI}	Claude-Sonnet-3.5	248	7.3	0.86
AUTOCODEROVER	GPT-4o	192	-	0.65
AUTOCODEROVER _{ADI}	GPT-4o	204	6.2	0.72

Subscript "ADI" denotes the approach integrated with Agent-centric Debugging Interface.

is substantially higher (e.g., \$0.60). This inefficiency is rooted in the nature of traditional debugging. For instance, when using Claude-Sonnet-3.7, BaseAgent_{pdb} invokes the next command an average of 10.1 times in 191 tasks where it attempts debugging. Our manual analysis of all 191 of these attempts reveals that this inefficient process leads the agent to abandon its attempt in 53% of them.

Finding 1: Agent-centric Debugging Interface advances the capabilities of autonomous agents for automated program repair tasks. It enables a basic agent to achieve superior performance on SWE-bench, even slightly outperforming highly-optimized and high-investment tools while also solving tasks they cannot at a low cost.

The strong performance and efficiency gains of ADI on a baseline agent motivate an inquiry into its generality. In RQ2, we then investigate whether ADI can serve as a plug-and-play enhancement for the existing state-of-the-art agents.

5.4.2 RQ2: Generalizability to SOTA Agents. As shown in Table 3, integrating Agent-centric Debugging Interface as a plug-and-play component consistently enhances the performance of both mini-SWE-agent and AUTOCODEROVER. Across different LLMs, this integration delivers performance gains ranging from 6.2% to 18.5% at a modest cost increase. Specifically, for mini-SWE-agent, whose ReAct-based workflow relies on binary pass/fail signals, ADI provides a crucial enhancement. It equips the agent with the capability to inspect the program's internal state upon a failure, enabling targeted, diagnostic reasoning that goes beyond a simple test outcome. This enhanced reasoning capability boosts the number of resolved tasks by 10.6% with Claude-Sonnet-3.7 and 18.5% with GPT-4o. Similarly, ADI augments AUTOCODEROVER's static analysis-based strategy by providing dynamic runtime information. This richer, hybrid dynamic context empowers the agent to generate more precise patches for bugs only observable at runtime, resulting in resolved rate improvements of 7.3% with Claude-Sonnet-3.5 and 6.2% with GPT-4o. All such results demonstrate that ADI could enhance the performance of general agent architectures for automated program repair tasks.

Finding 2: Agent-centric Debugging Interface acts as a general-purpose enhancement, consistently improving the performance of agents with distinct architectures at modest costs.

5.4.3 RQ3: characteristics of Agent-centric Debugging Interface. We first investigate the characteristics of ADI by analyzing its invocation rate on the SWE-bench Lite and Verified benchmarks. Figure 8 reveals a strong correlation between the agent's utilization of the ADI and the capability of its underlying LLM. For instance, with Claude-Sonnet-3.7, the agent invokes ADI commands

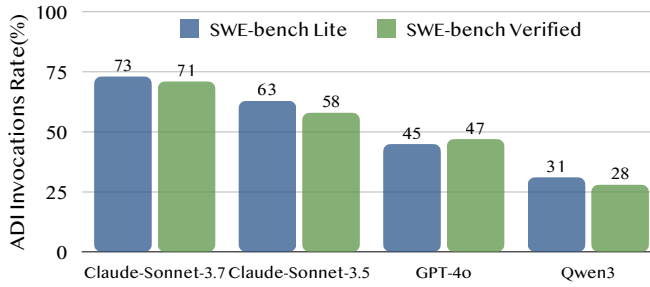


Fig. 8. Invocation rate of ADI on the SWE-bench Lite and Verified benchmarks.

in 73% of tasks on SWE-bench Lite and 71% on SWE-bench Verified. In contrast, this rate drops significantly to just 31% and 28% for Qwen3. Meanwhile, Table 2 shows that on SWE-bench Verified, the resolved rate of BaseAgent descends from 55.0% for Claude-Sonnet-3.7 to 29.2% for Qwen3. Such a trend aligns with the above-mentioned ADI invocation rates, indicating that stronger LLMs are more adept at leveraging ADI, allowing them to amplify their inherent analytical capabilities. For the 62 tasks uniquely resolved by FRAMEPILOT over BaseAgent on SWE-bench Verified with Claude-Sonnet-3.7, ADI commands are invoked 6.39 times per task on average.

Moreover, we find that `break` is the most frequently-used command, invoked an average of 2.5 times, followed by `continue` with an average of 2.4 invocations, across the four LLMs used by FRAMEPILOT. Interestingly, when analyzing the FRAMEPILOT with Claude-Sonnet-3.7, we observe that the `execute` command is invoked in 44 tasks with an average of 3.1 invocations each, while the `call-tree` command is used in 16 tasks, averaging 1.0 invocations.

Agent: Let's check the autoescape settings for this last call:

```
<function_call>
<function>dbg</function>
<action>execute</action>
<cmd>python /testbed/test_fix_verification.py</cmd>
<id>/testbed/django/template/base.py:Template.render#8</id>
<stmt>print(f"Context autoescape setting: {context.autoescape}, Engine autoescape setting:
{self.engine.autoescape}")</stmt>
<lineno>167</lineno>
</function_call>
```

[Result]: Context autoescape setting: True, Engine autoescape setting: False

```
Frame: /testbed/django/template/base.py:Template.render#8
Line: 167
Statement: print(f"Context autoescape setting: {context.autoescape}, Engine
autoescape setting: {self.engine.autoescape}")
[execute] Execution completed
```

Agent: **Now I understand the issue completely.** The problem is that when creating a Context object in the Template class, it's not using the engine's autoescape setting. This happens in two places...

Fig. 9. Diagnosing the state mismatch in django-11119 [6].

We present a case study on the django-11119 task [6], which involves a subtle, non-crashing bug where the template Engine fails to apply its autoescape setting, causing the template variables to be rendered without proper HTML escaping. This type of state-dependent issue is exceptionally

difficult to diagnose with coarse-grained, post-mortem analysis, as the final output offers no clue about the internal state discrepancy. After navigating to the suspected function call, FRAMEPILOT leverages the `execute` command to inject a diagnostic statement into a specific execution frame (`Template.render#8`), allowing it to simultaneously inspect the `autoescape` attribute of both the local `Context` object and the parent `Engine` object. As shown in Figure 9, this single, targeted query immediately revealed the critical state mismatch (`Context autoescape: True` vs. `Engine autoescape: False`), providing a precise fault localization that would be more costly and complex with traditional line-by-line debugging.

Finding 3: The utilization of ADI scales with LLM capability, as stronger models invoke the interface more frequently.

Table 4. *Frame Lifetime Trace* component ablation on the SWE-bench Lite benchmark

FLT component	#Resolved	Avg.\$
Basic (BaseAgent)	130	0.90
Basic+ τ	142	1.14
Basic+ τ +args	145	1.23
Basic+ τ +args+ret (FRAMEPILOT)	152	1.26

We further perform an ablation study on the components of our designed *Frame Lifetime Trace*. Specifically, we evaluate four settings on the SWE-bench Lite benchmark using Claude-Sonnet-3.7, starting from a baseline BaseAgent and progressively adding *Frame Lifetime Trace* components (τ , *args*, and *ret*) to form the full FRAMEPILOT. As shown in Table 4, each component contributes to the performance, with the number of resolved tasks increasing from 130 for the BaseAgent to 152 for the FRAMEPILOT under a moderate average cost increase from \$0.90 to \$1.26.

6 Threats to Validity

Threats to internal validity. We take several measures to ensure the internal validity of our Agent-centric Debugging Interface. First, we address the potential confounding effect of runtime overhead from our tracing mechanism. By using a selective instrumentation strategy, we limit tracing only to the function under inspection. Our measurements confirm this approach is effective, showing only a modest increase in average test execution time (0.68s to 0.87s) on the SWE-bench Verified set. A potential threat to internal validity arises from Agent-centric Debugging Interface’s on-demand tracing design, which requires re-execution and may therefore introduce additional runtime overhead and sensitivity to non-deterministic executions. In practice, Agent-centric Debugging Interface re-executes the program 5.19 times per task on average, corresponding to about 4 seconds of total execution time per task on SWE-bench Verified. To mitigate this threat, we prompt the agent to generate minimal reproduction scripts as stable debugging entry points and restrict Agent-centric Debugging Interface from tracing external libraries; we did not observe noticeable non-deterministic effects in our evaluation. Then, to guarantee reproducibility and mitigate system-level variations, all evaluations are conducted using the official SWE-bench Docker environment, which encapsulates the exact repository state and dependencies for each task. To address the inherent stochasticity of LLMs, we set the model’s temperature to 0 for all experiments. Furthermore, our evaluation is performed on large-scale benchmarks (SWE-bench Lite and Verified, with 300 and 500 tasks respectively), ensuring that our findings are stable. Any manual analysis, such as the case studies, was cross-validated by three authors to mitigate potential interpretation bias.

Threats to external validity. The primary threat to external validity lies in the generalizability of our ADI and findings. We mitigate this by grounding our evaluation in the SWE-bench dataset, a widely-recognized benchmark composed of real-world software engineering tasks from popular open-source Python repositories. The authenticity of these tasks ensures our results reflect challenges faced in genuine development scenarios. Moreover, for our generality study (RQ2), we integrate ADI into two representative, state-of-the-art agents (mini-SWE-agent and AUTOCODEROVER) chosen for their distinct architectures. This demonstrates that the benefits of ADI are not confined to our specific agent implementation but can extend to other existing frameworks. However, the generalizability of ADI may still vary across bug classes. In particular, ADI is currently better suited to bugs whose diagnosis depends on function-level data flow, control flow, and state mutations, while bugs involving concurrency effects, memory leaks, or interactions spanning many shallow calls may be less well supported by the current FLT abstraction. Moreover, as a modular debugging interface, ADI may also depend in part on how effectively a host agent invokes it. Our cross-agent evaluation on mini-SWE-agent and AUTOCODEROVER partially mitigates this concern. Additionally, although our current implementation is validated on Python, the core idea of ADI is not inherently Python-specific, as the stack frame is a fundamental abstraction across many programming languages and can be extended to other language ecosystems with appropriate instrumentation support.

Threats to construct validity. A potential threat to construct validity is whether our evaluation metrics accurately capture an agent's problem-solving capability. To address this, our primary metric is the Resolved Rate (Pass@1), the standard and most accepted metric for task completion in this domain. We complement this with two diagnostic metrics: the Correct Location Rate (%) to assess the precision of the generated patches, and the Average Cost (\$) to measure the economic efficiency of the process. Collectively, these metrics provide a holistic assessment, evaluating not only the final correctness but also the accuracy and efficiency of the solution.

7 Conclusion

In this paper, we address the limitations of autonomous agents, which are hindered by coarse-grained execution feedback and the cost-inefficiency of traditional line-by-line debuggers. We introduce Agent-centric Debugging Interface, a novel agent-centric debugging interface that enables cost-efficient dynamic analysis through a function-level interaction model, powered by our *Frame Lifetime Trace* and high-level navigational commands. Our evaluation on SWE-bench demonstrates that a basic agent equipped with Agent-centric Debugging Interface achieves superior performance, resolving 63.8% of tasks on the SWE-bench Verified benchmark at a low average cost. Finally, we establish Agent-centric Debugging Interface as a general enhancement, showing that it delivers consistent performance gains when integrated as a plug-and-play component into the existing agents.

Data Availability

The source code of our Agent-centric Debugging Interface, along with all agent implementations, evaluation scripts, and detailed results, are publicly available at our GitHub repository [7, 55].

Acknowledgments

This work is partially supported by the National Natural Science Foundation of China (Grant No. 62372220). It is also partially supported by Ant Group Research Fund.

References

- [1] 2024. *Whether using test patch is allowed*. <https://github.com/swe-bench/experiments/issues/16> GitHub issue #16, SWE-bench/experiments.
- [2] 2024-02-29. Hugging Face. <https://huggingface.co>.
- [3] 2024-02-29. OpenAI API. <https://openai.com/api>.
- [4] 2025. astropy-12907 github issue. <https://github.com/astropy/astropy/issues/12906>.
- [5] 2025. Astropy Github Repository. <https://github.com/astropy/astropy>.
- [6] 2025. django-11119 task pr. <https://github.com/django/django/pull/11119/>.
- [7] 2025. Github Repository. <https://github.com/GhabiX/ADI>.
- [8] Talor Abramovich, Meet Udeshi, Minghao Shao, Kilian Lieret, Haoran Xi, Kimberly Milner, Sofija Jancheska, John Yang, Carlos E Jimenez, Farshad Khorrami, et al. 2024. Enigma: Enhanced interactive generative model agent for ctf challenges. *arXiv preprint arXiv:2409.16165* (2024).
- [9] Rui Abreu, Peter Zoetewij, and Arjan JC Van Gemund. 2007. On the accuracy of spectrum-based fault localization. In *Testing: Academic and industrial conference practice and research techniques-MUTATION (TAICPART-MUTATION 2007)*. IEEE, 89–98.
- [10] Alibaba Cloud. 2025. Qwen3 API Price. Online. <https://www.alibabacloud.com/help/zh/model-studio/models> Accessed: 2025-07-12.
- [11] Anthropic. 2024. Introducing Claude 3.5 Sonnet. Online. <https://www.anthropic.com/news/claude-3-5-sonnet> Accessed: 2025-07-15.
- [12] Anthropic. 2025. Claude 3.7 Sonnet and Claude Code. Online. <https://www.anthropic.com/news/claude-3-7-sonnet> Accessed: 2025-07-15.
- [13] Anthropic. 2025. Claude API Documentation. <https://docs.anthropic.com/en/home>. Accessed: 2025-06-30.
- [14] Anthropic. 2025. Raising the Bar on SWE-Bench Verified with Claude 3.5 Sonnet. <https://www.anthropic.com/engineering/swe-bench-sonnet>. Accessed: 2025-09-11.
- [15] Yasharth Bajpai, Bhavya Chopra, Param Biyani, Cagri Aslan, Dustin Coleman, Sumit Gulwani, Chris Parnin, Arjun Radhakrishna, and Gustavo Soares. 2024. Let’s fix this together: Conversational debugging with github copilot. In *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 1–12.
- [16] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [17] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).
- [18] Islem Bouzenia, Yangruibo Ding, Kexin Pei, Baishakhi Ray, and Michael Pradel. 2023. Tracefixer: Execution trace-driven program repair. *arXiv preprint arXiv:2304.12743* (2023).
- [19] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).
- [20] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*. 423–435.
- [21] Konstantin Grotov, Artem Borzilov, Maksim Krivobok, Timofey Bryksin, and Yaroslav Zharov. 2024. Debug smarter, not harder: Ai agents for error resolution in computational notebooks. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 363–371.
- [22] Yirui He, Ziyao He, Syed Fatiul Huq, and Sam Malek. 2026. ReFLAIR: Detecting Responsive Layout Reflow Issues using Multimodal Generative AI. *Proceedings of the ACM on Software Engineering* 3, FSE (2026). doi:10.1145/3808136
- [23] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [24] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [25] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2025. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering* 30, 2 (2025), 45.
- [26] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
- [27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

- [28] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 919–931.
- [29] Kyla H Levin, Nicolas van Kempen, Emery D Berger, and Stephen N Freund. 2025. ChatDBG: Augmenting Debugging with Large Language Models. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1892–1913.
- [30] Hongwei Li, Yuheng Tang, Shiqi Wang, and Wenbo Guo. 2025. PatchPilot: A Stable and Cost-Efficient Agentic Patching Framework. *arXiv preprint arXiv:2502.02747* (2025).
- [31] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2023), 42330–42357.
- [32] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977* (2024).
- [33] Zhengyao Liu, Yunlong Ma, Jingxuan Xu, Junchen Ai, Xiang Gao, Hailong Sun, and Abhik Roychoudhury. 2025. Agent That Debugs: Dynamic State-Guided Vulnerability Repair. *arXiv preprint arXiv:2504.07634* (2025).
- [34] Yingwei Ma, Rongyu Cao, Yongchang Cao, Yue Zhang, Jue Chen, Yibo Liu, Yuchen Liu, Binhua Li, Fei Huang, and Yongbin Li. 2024. Lingma swe-gpt: An open development-process-centric language model for automated software improvement. *arXiv preprint arXiv:2411.00622* (2024).
- [35] OpenAI. 2024. Hello GPT-4o. Online. <https://openai.com/index/hello-gpt-4o/>. Accessed: 2025-07-15.
- [36] OpenAI. 2024. Introducing SWE-bench, verified. <https://openai.com/index/introducing-swe-bench-verified/>. Accessed on 2025-06-23.
- [37] Python Docs. 2025. Python ctypes. <https://docs.python.org/3/library/ctypes.html>.
- [38] Python Docs. 2025. Python Frame Objects. <https://docs.python.org/3/c-api/frame.html>.
- [39] Python Software Foundation. 2023. Python 3 Glossary — qualified name. <https://docs.python.org/3/glossary.html#term-qualified-name>.
- [40] Python Software Foundation. 2025. *pdb* — The Python Debugger. <https://docs.python.org/3/library/pdb.html>
- [41] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. 2025. Specrover: Code intent extraction via llms. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 963–974.
- [42] Richard Stallman, Roland Pesch, Stan Shebs, et al. 1988. Debugging with GDB. *Free Software Foundation* 675 (1988).
- [43] Hanzhuo Tan, Weihao Li, Xiaolong Tian, Siyi Wang, Jiaming Liu, Jing Li, and Yuqun Zhang. 2025. SK2Decompile: LLM-based Two-Phase Binary Decompile from Skeleton to Skin. *arXiv preprint arXiv:2509.22114* (2025).
- [44] Hanzhuo Tan, Qi Luo, Ling Jiang, Zizheng Zhan, Jing Li, Haotian Zhang, and Yuqun Zhang. 2024. Prompt-based code completion via multi-retrieval augmented generation. *ACM Transactions on Software Engineering and Methodology* (2024).
- [45] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. 2024. Llm4decompile: Decompiling binary code with large language models. 3473–3487 pages.
- [46] Hanzhuo Tan, Xiaolong Tian, Hanrui Qi, Jiaming Liu, Siyi Wang, GAO Zuchen, Qi Luo, Jing Li, and Yuqun Zhang. [n. d.]. Decompile-Bench: Million-Scale Binary-Source Function Pairs for Real-World Binary Decompilation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [47] Hanzhuo Tan, Chunpu Xu, Jing Li, Yuqun Zhang, Zeyang Fang, Zeyu Chen, and Baohua Lai. 2024. Hicl: Hashtag-driven in-context learning for social media natural language understanding. *IEEE transactions on neural networks and learning systems* 36, 4 (2024), 7037–7050.
- [48] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. A Study on Developer Behaviors for Validating and Repairing LLM-Generated Code Using Eye Tracking and IDE Actions. *arXiv preprint arXiv:2405.16081* (2024).
- [49] The SWE-bench Team. 2024. SWE-bench: A Benchmark for Evaluating Large Language Models on Real World Software Issues. <https://www.swebench.com>. Accessed: 28-June-2025.
- [50] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741* (2024).
- [51] Maurice V. Wilkes, David J. Wheeler, and Stanley Gill. 1951. *The Preparation of Programs for an Electronic Digital Computer*. Addison-Wesley Press, Cambridge, MA, USA.
- [52] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).
- [53] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
- [54] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing*

and Analysis. 819–831.

- [55] Jiahong Xiang. 2026. FramePilot-Artifacts. [doi:10.5281/zenodo.19728388](https://doi.org/10.5281/zenodo.19728388)
- [56] Jiahong Xiang, Wenxiao He, Xihua Wang, Hongliang Tian, and Yuqun Zhang. 2026. Evaluating and Improving Automated Repository-Level Rust Issue Resolution with LLM-based Agents. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*. IEEE, 816–830. [doi:10.1145/3744916.3773108](https://doi.org/10.1145/3744916.3773108)
- [57] Jiahong Xiang, Xiaoyang Xu, Fanchu Kong, Mingyuan Wu, Zizheng Zhang, Haotian Zhang, and Yuqun Zhang. 2024. How far can we go with practical function-level program repair? *arXiv preprint arXiv:2404.12833* (2024).
- [58] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [59] John Yang, Kilian Leret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798* (2025).
- [60] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv:2210.03629*
- [61] Xingdi Yuan, Morgane M Moss, Charbel El Feghali, Chinmay Singh, Darya Moldavskaya, Drew MacPhee, Lucas Caccia, Matheus Pereira, Minseon Kim, Alessandro Sordani, et al. 2025. debug-gym: A Text-Based Environment for Interactive Debugging. *arXiv preprint arXiv:2503.21557* (2025).
- [62] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 39–51.

Received 2025-09-12; accepted 2025-12-22